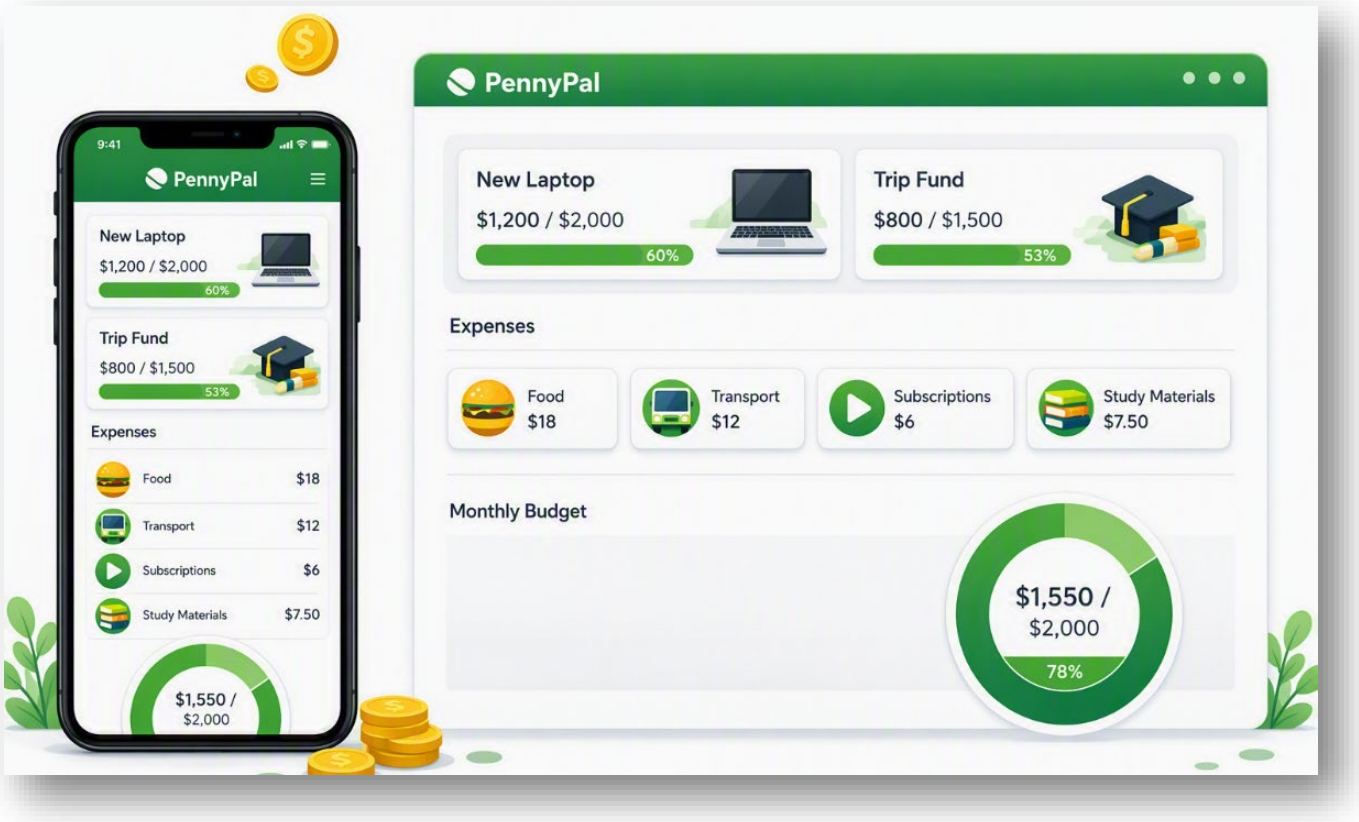


PennyPal

Fresh All Along

Software Requirements Specification

Version 1.0

Project Name: PennyPal

Theme: NextGen BudgetBee

Category: Multi-Platform App Computing

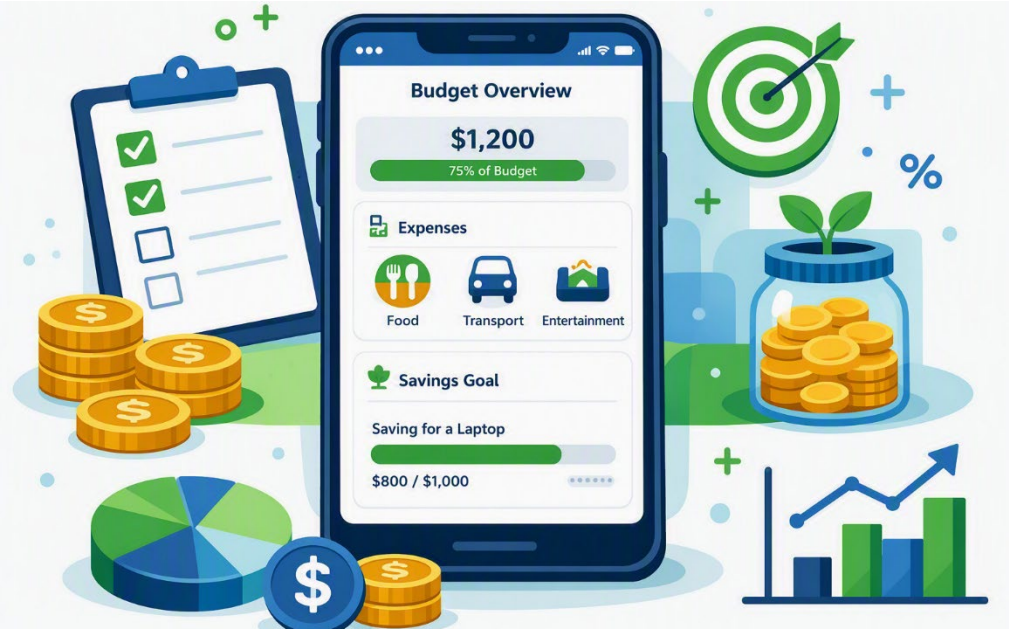
Table of Contents

1.1 Background and Necessity for the Cross-Platform App 3
1.2 Proposed Solution 3
1.3 Purpose of the Document 4
1.4 Scope of Project 5
1.5 Constraints 6
1.6 Functional Requirements 8
1.7 Non-Functional Requirements 12
1.8 Interface Requirements 14
1.9 Project Deliverables 15

1.1 Background and Necessity for the Cross-Platform App

Financial literacy is an important life skill, but many students start receiving money from allowances, scholarships, internships, or part-time jobs without knowing how to record their expenses or make a simple budget. Daily expenses such as food, transport, entertainment, subscriptions, shopping, and study materials can quickly use up their money and leave less for important necessities and future savings.

Traditional methods such as notebooks and spreadsheets are often time-consuming and inconvenient for students who require a quick way to record expenses and understand their spending habits. A cross-platform application can provide a more practical solution by combining expense tracking, budget planning, savings support, and financial education in one platform that works across mobile and Web enabled devices.



1.2 Proposed Solution

A responsive cross-platform application named **PennyPal** will be developed that helps students manage their money more effectively. It will allow students to record expenses, set budgets, track savings goals, review spending patterns, and improve their understanding of personal finance. The application will be accessible on Android phones, iPhones, and tablets. A shared Flutter or Android codebase will be used to provide consistent experience across all supported platforms.

The application should enable students to:

- Record expenses manually.
- Set monthly budgets and category-wise spending limits.
- Receive instant notifications when spending approaches defined thresholds.
- Create savings goals and track progress.
- Attach camera-captured images to transactions for future reference (optional feature).
- Utilize AI-powered expense categorization (optional feature).
- Interact with an AI chatbot for budgeting advice and spending-related queries.
- View detailed financial reports and spending trends.

PennyPal is intended as a personal finance-learning and expense-management application. It must not claim to be a bank, digital wallet, payment gateway, investment platform, or professional financial advisory service. It must not initiate real financial transactions.

1.3 Purpose of the Document

This document outlines the complete software requirements for the PennyPal cross-platform application. It explains the intended user roles, core features, technical scope, data structure, quality standards, interface requirements, and project submission deliverables.

It also covers the Administrator role, that is responsible for managing student users, learning content, app settings, support queries, and overall application monitoring.

This document defines

- Scope
- Functional and non-functional requirements
- User roles and permissions
- Database entities
- Implementation deliverables

1.3.1 Who Can Use This Document?

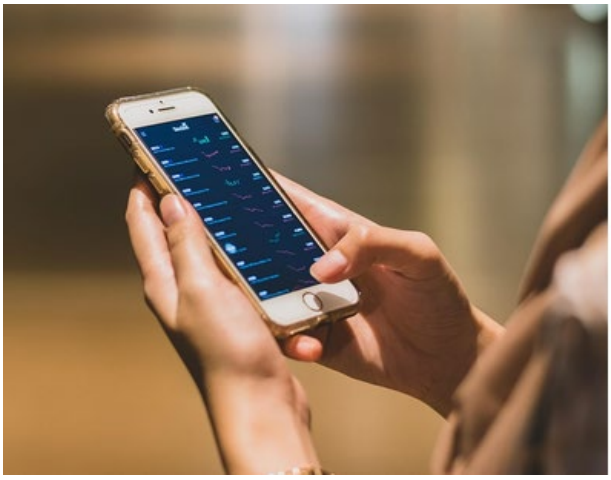
This document helps the development team confirm which features must be implemented and which financial services are intentionally excluded from the project.

This document can be used by	Students
	Trainers
	Evaluators
	Project reviewers
	Designers
	Developers
	Testers
	Project Managers

1.4 Scope of Project

PennyPal is designed as a cross-platform personal finance application for students and beginner students who want a simple, guided way to manage everyday money. The application will support expense tracking, budget planning, savings goal monitoring, spending insights, learning content, and basic AI-assisted financial guidance. It will work on Android phones, iOS, and tablets using a consistent interface and shared application logic.

In addition to student-facing features, the project includes a limited Administrator role for maintaining app content, reviewing usage summaries, managing support queries, and ensuring that the application remains organized and reliable.



Students Features	Administrators Features
• Track daily expenses • Log expenses • Categorize spending automatically using AI (optional feature) • Create monthly budgets • Set spending limits for different categories • Create savings goals • Monitor financial progress • Receive spending and savings notifications • Generate spending reports • Chat with an AI assistant for financial guidance	• Manage student users • Monitor application activity • Generate analytics reports • Manage educational content and financial literacy resources for Students

1.5 Constraints

The application must remain within the scope of a student personal finance management system. It should not connect to real bank accounts, perform payments, store card details, provide investment recommendations, or claim to replace professional financial advice. Any examples, tips, or AI responses must be educational and clearly presented as guidance only.

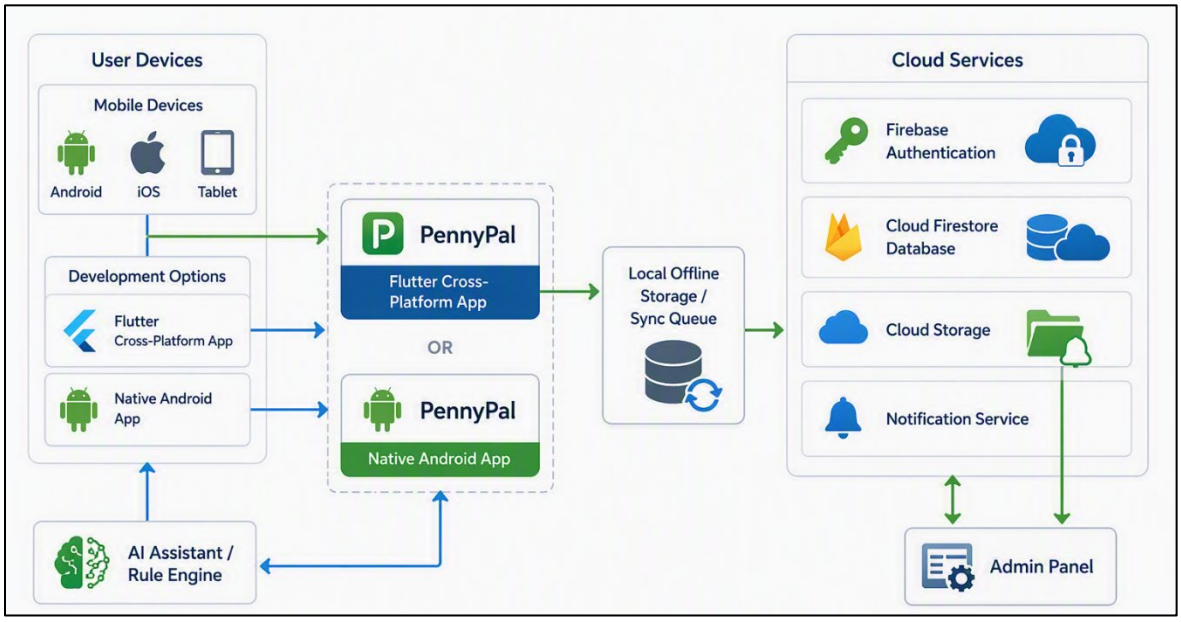
To ensure smooth operation, PennyPal should satisfy following constraints:

Constraint	Description
Real-Time Processing	Generate spending alerts instantly when students approach category limits.
Cross-Platform Compatibility	Ensure consistent performance across Android, iOS, tablets, and Web browsers.
Security and Privacy	Protect financial data using encryption and secure authentication.
Offline Support	Allow offline expense entry and automatically sync data when connectivity is restored.
AI Response Latency	AI-generated insights and chatbot responses should return within a few seconds; the app should show a loading state and fail gracefully to a static/default view if the API call times out or fails.

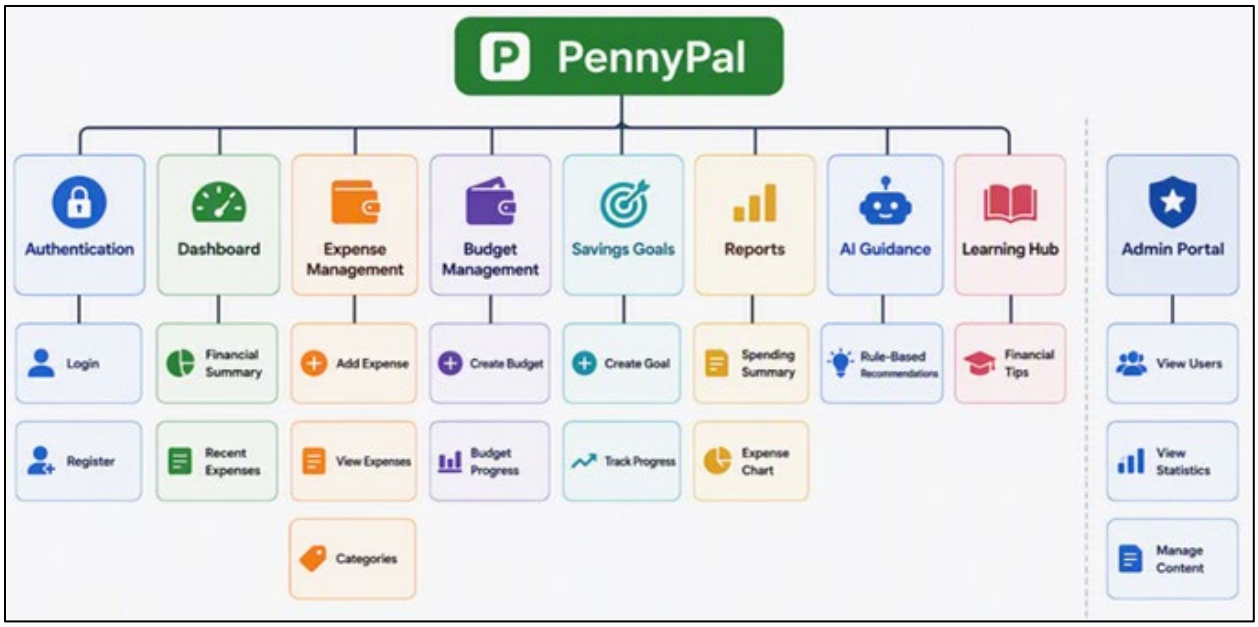
Architecture Diagram

The broad architecture of PennyPal is based on a cross-platform client application that communicates with cloud services for authentication, data storage, file storage, notifications, and AI-assisted learning support The application also maintains a local offline cache so students can record expenses during poor connectivity and synchronize records when the network is restored.

A sample broad architecture is shown here.



Sample Site Map



Boilerplate or readymade HTML template can be used, provided it is only for design aspect and not for implementing app functionality. Do NOT copy content or code from GPTs or other AI tools, although you are permitted to use images generated by AI tools for any visual representation purposes. It is mandatory to mention such tools used in case you add any AI generated images.

1.6 Functional Requirements

User Registration and Authentication (except Admin whose logins are preconfigured)



- The app should allow students to register using name, email, mobile number, and password.
- Students should be able to log in securely using email and password.
- The app should validate required fields, email format, password rules, and duplicate accounts.
- Students should be able to log out from the application.

Dashboard

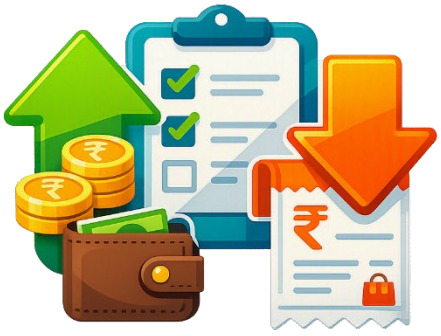


- The dashboard should display total income, total expenses, available balance, and budget status.
- The dashboard should provide quick navigation to Add Income, Add Expense, Budget, Transaction History, Learning, Feedback, and Contact Support screens.
- The dashboard should show simple visual indicators such as cards, progress bars, or basic charts for income, expense, and budget summary.
- The dashboard should display a clear message when no income, expense, or budget data is available.

Income and Expense Management

- Students should be able to add income entries with source, amount, date, and description.
- Students should be able to add expense entries with category, amount, date, and description.
- Students should be able to view, edit, and delete income and expense records.
- Expense categories should include Food, Transport, Education, Shopping, Entertainment, Bills, Savings, and Miscellaneous.
- The transaction history should support basic search and filter by date, category, and transaction type.

Budget Management



- Students should be able to create a monthly budget with a total budget amount.
- The app should compare monthly expenses with the set budget.
- The app should show remaining budget and overspending information clearly.
- Students should be able to update or delete an existing budget.

Learning and Tips

- The app should include a learning section with simple finance topics such as budgeting, saving, income, necessary expenses, requirements, and optional expenses.
- The learning content should be displayed through short text, cards, examples, or simple images.
- Students should be able to open and read individual learning topics from the learning screen.

About, Feedback, and Contact Support



- The app should include an About screen explaining the purpose of PennyPal.
- The Feedback screen should allow students to submit name, email, rating, and comments.
- The Contact Support screen should allow students to submit subject and message details.
- The app should validate required fields and show a confirmation message after successful submission.
- The app should include a chatbot where students can ask simple budgeting or savings-related questions and receive educational guidance.

Savings Goals

- Students should be able to create savings goals with goal name, target amount, current savings, target date, and monthly contribution
- The app should calculate remaining amount, progress percentage, and estimated completion time
- Students should be able to update contribution amounts and mark milestones
- The app should display goal progress using charts, progress bars, badges, or motivational messages
- Completed goals should be archived and visible in goal history

Important	Do not use boilerplate or readymade app templates for complete functionality. The project must demonstrate the student's own cross-platform design, database planning, UI logic, validation, and feature implementation skills. Third-party libraries may be used only where appropriate and should be acknowledged in the project documentation.

Important Note Regarding AI Usage:

You are encouraged to use AI-powered tools (such as AI-powered UI/UX design tools, code assistants, and image-generation tools) to enhance productivity and creativity. However, AI should be used as a supporting aid rather than a substitute for your own design, development, and problem-solving skills.

Do NOT rely on completely ready-made mobile templates for your project, as this will adversely affect your evaluation. Your Web application's design, structure, and implementation should primarily reflect your own skills and understanding. Do NOT submit AI-generated code or content without meaningful modification and understanding. AI-generated suggestions may be used for guidance, learning, debugging, or improving productivity, but the final solution should demonstrate your own effort, logic, and implementation.

Acknowledge all the AI tool(s) used (for example, Copilot, Canva AI, Figma AI, Uizard, or similar) in your project documentation or submission.

During evaluation, judges may ask participants to explain their design decisions, implementation approach, and code. You are, therefore, expected to understand and be able to justify all aspects of your submitted work.

Do not use AI tools to fully produce ready-made documentation. This is strictly forbidden.

Bottomline: AI is your **assistant**, not your **developer**. Your knowledge, creativity, and coding skills should drive the project.

1.7 Non-Functional Requirements

Following table lists the key non-functional requirements that PennyPal should satisfy to ensure quality, usability, security, and reliability.

Requirement	Description
Safety	The app should not trigger malicious downloads, unwanted actions, or unsafe external redirects
Accessibility	Text, controls, charts, images, forms, and navigation should be readable, labelled, and usable with assistive support wherever possible
User-Friendliness	The app should use simple language, clear menus, helpful validation, and beginner-friendly finance terms
Performance	Screens should load quickly, dashboards should refresh smoothly, and transaction entry should remain responsive
Reliability	The app should handle invalid inputs, network failures, duplicate sync attempts, and incomplete records without crashing
Security	Authentication, access control, secure storage, and encrypted transmission should be implemented for user data
Privacy	The app should avoid collecting unnecessary sensitive data and should clearly explain how user financial records are used
Scalability	The architecture should support more student users, categories, tips, reports, and future modules without major redesign
Compatibility	The app should work on Android, iOS, tablets, and modern Web browsers
Maintainability	Code, assets, database rules, and documentation should be organized, commented, and easy to update

Database Design

The application may use Firebase for cloud data storage and SQLite or an equivalent local database for offline caching.

Following entities may be used as a sample/reference design and can be modified during implementation:

Entity	Important Attributes
Users	UserId(PK), FullName, Email, MobileNumber, PasswordHash, Role, CreatedAt, LastLogin
UserProfiles	ProfileId(PK), UserId(FK), StudentStatus, CurrencyPreference, NotificationPreference, ProfileImage
Transactions	TransactionId(PK), UserId(FK), Type, Amount, CategoryId(FK), Description, Date, PaymentMode, ReceiptImageUrl, CreatedAt
Categories	CategoryId(PK), CategoryName, CategoryType, Icon, IsDefault, CreatedBy(FK)
Budgets	BudgetId(PK), UserId(FK), Month, CategoryId(FK), LimitAmount, AlertThreshold, CreatedAt
SavingsGoals	GoalId(PK), UserId(FK), GoalName, TargetAmount, CurrentAmount, TargetDate, MonthlyContribution, Status
Reports	ReportId(PK), UserId(FK), ReportType, DateRange, GeneratedOn, FileUrl
LearningContent	ContentId(PK), Title, Topic, Body, DifficultyLevel, ImageUrl, IsActive
Notifications	NotificationId(PK), UserId(FK), Title, Message, Type, ReadStatus, CreatedAt
SupportQueries	QueryId(PK), UserId(FK), Subject, Message, Status, AdminResponse, SubmittedOn

1.8 Interface Requirements

Hardware Requirements	Software Requirements
• Development system with Intel Core i5/i7 or equivalent processor • 8 GB RAM or higher • At least 500 GB free storage for project files and assets • Android or iOS device or emulator for app testing • Internet connection for cloud sync, authentication, and deployment testing	• **IDE:** Android Studio IDE with Android 9 or higher with Java OR Flutter 3.24, Dart 3.6 or higher and related libraries • **AI tools:** AI-related Dart packages (such as dart_ml, string_similarity, ml_linalg), TensorFlow Lite, Google ML Kit, Firebase ML Kit, Google AI Studio for API key creation • **Database:** SQLite or Firebase
Testing	
• You can test the cross-platform app using a mobile device (Android Smartphone or iPhone) or a suitable emulator. • **Testing on Android emulator, iOS simulator, physical device, and Web browser where applicable**	

1.9 Project Deliverables

You are required to design and build the project and submit it along with complete Documentation that includes:

- Problem Definition
- Design Specifications
- Diagrams such as Flowcharts for various Activities, Data Flow Diagrams, and so on
- Database Design
- Test Data Used in the Project
- **Project Installation Instructions (MANDATORY)**
- **User Credentials (Login ID and Password- MANDATORY)**

Documentation is considered as a very important part of the project. Ensure that documentation is complete and comprehensive.

Documentation should not contain any source code.

The consolidated project will be submitted as a Source Code zip file with a ReadMe.doc file listing assumptions (if any) made at your end. It should include SQL scripts files (.sql) containing database and table definitions and Android Package File (.apk file) for the cross-platform app.

Submit a video (.mp4 file) demonstrating the working of the cross-platform app, including all the functionalities of the project. This is MANDATORY.

Over and above the given specifications, you can apply your creativity and logic to improve the cross-platform app.

~~~ End of Document ~~~